■ Java String Functions

Complete Notes for Placements & Interviews

All Methods • Use Cases • Code Examples • Tricks

■■ 3 Golden Rules (Yaad Rakho!)

1. String comparison mein **always equals()** use karo — kabhi **==** mat use karo.
2. Loop mein String concatenation mat karo — **StringBuilder** use karo (fast & memory efficient).
3. Java mein String **immutable** hoti hai — har modify par naya object banta hai.

01 length()

Returns the total number of characters in the string.

```
String s = "hello";

System.out.println(s.length());  // Output: 5
```

■ Use Case: String size check karna — loop chalana, input validation.

02 charAt(index)

Returns the character at the specified index (0-based).

```
String s = "hello";

s.charAt(0);  // h

s.charAt(4);  // o
```

■ Use Case: Har ek character access karna — palindrome check, frequency count.

03 indexOf() / lastIndexOf()

indexOf() returns first occurrence index of substring. lastIndexOf() returns last occurrence. Returns -1 if not found.

```
String s = "hello world";

s.indexOf("world");     // 6

s.indexOf("xyz");       // -1  (not found)

s.lastIndexOf("l");     // 9
```

■ Use Case: Substring kahan hai dhundhna. -1 matlab not found — safe way to search.

04 substring(start, end)

Returns a portion of the string. substring(start) returns from index to end. substring(start, end) returns start to end-1.

```
String s = "hello world";

s.substring(6);      // "world"

s.substring(0, 5);   // "hello"
```

■ **Use Case: String ka specific part nikalna — slicing jaisa kaam Java mein.**

05 toUpperCase() / toLowerCase()

Converts entire string to uppercase or lowercase. Original string unchanged (immutable).

```
String s = "Hello World";

s.toUpperCase();  // "HELLO WORLD"

s.toLowerCase();  // "hello world"
```

■ **Use Case: Case-insensitive comparison — username/password match, search.**

06 trim()

Removes leading and trailing whitespace (spaces, tabs, newlines). Does not remove spaces in the middle.

```
String s = "  hello  ";

s.trim();  // "hello"


// strip() is modern version (Java 11+)

"  hello  ".strip();  // "hello"
```

■ **Use Case: User input se extra spaces hatana — form validation, data cleaning.**

07 replace() / replaceAll()

replace() replaces literal characters/strings. replaceAll() supports regex patterns.

```
String s = "I love Java";

s.replace("Java", "Python");      // "I love Python"

s.replaceAll("[aeiou]", "*");      // replaces all vowels

s.replace('l', 'L');              // char replace
```

■ Use Case: Word replace karna, regex se pattern replace — text cleaning problems.

08 contains()

Returns true if the string contains the specified sequence. Case-sensitive.

```
String s = "hello world";

s.contains("world");  // true

s.contains("xyz");    // false

s.contains("World");  // false (case sensitive)
```

■ Use Case: Substring exist karta hai check karna — search, filter problems.

09 startsWith() / endsWith()

Checks if string begins or ends with specified prefix/suffix. Returns boolean.

```
String s = "hello.java";

s.startsWith("hello");  // true

s.endsWith(".java");    // true

s.startsWith("world"); // false
```

■ Use Case: File extension check, URL validation, prefix matching problems.

10 equals() / equalsIgnoreCase()

equals() compares content exactly. equalsIgnoreCase() ignores case. NEVER use == for String comparison in Java!

```
String s1 = "Hello";

String s2 = "hello";

s1.equals(s2);              // false

s1.equalsIgnoreCase(s2);    // true


// WRONG way (compares reference, not content)

// s1 == s2  -- DO NOT USE
```

■ Use Case: String comparison — login validation, answer matching. Always use equals(), not ==.

11 compareTo()

Compares two strings lexicographically. Returns 0 if equal, negative if s1 comes before s2, positive if s1 comes after s2.

```
String s1 = "apple";

String s2 = "banana";

s1.compareTo(s2);    // negative (a before b)

"abc".compareTo("abc"); // 0 (equal)
```

■ **Use Case: Alphabetical sorting — used in sorting algorithms and comparators.**

12 split()

Splits the string around matches of the given regex/delimiter. Returns String array.

```
String s = "apple,banana,mango";

String[] arr = s.split(",");

// arr = ["apple", "banana", "mango"]


String sentence = "hello world";

String[] words = sentence.split(" ");

// words = ["hello", "world"]
```

■ **Use Case: CSV data parse karna, sentence ko words mein todna — very common!**

13 String.join()

Joins multiple strings with a delimiter. Opposite of split(). Static method.

```
String result = String.join(",", "apple", "banana", "mango");

// "apple,banana,mango"


// Join from List

List<String> list = Arrays.asList("a","b","c");

String.join("-", list);  // "a-b-c"
```

■ **Use Case: List of strings ko ek string mein milana — reverse words, combine tokens.**

14 toCharArray()

Converts string to a char array. Very useful for manipulation and sorting.

```
String s = "hello";

char[] arr = s.toCharArray();

// arr = ['h','e','l','l','o']


// Convert back

String back = new String(arr);  // "hello"
```

■ Use Case: String ko character array mein convert — sorting, anagram check, frequency count.

15 String.valueOf()

Converts other data types (int, char, float, boolean) to String.

```
int n = 123;

String s = String.valueOf(n);   // "123"


char c = 'A';

String cs = String.valueOf(c);  // "A"


boolean b = true;

String bs = String.valueOf(b);  // "true"
```

■ Use Case: int/char/float ko String mein convert karna — number to string problems.

16 Integer.parseInt() / toString()

parseInt() converts String to int. toString() converts int to String.

```
String s = "123";

int n = Integer.parseInt(s);    // 123


int x = 456;

String str = Integer.toString(x); // "456"


// Also: Double.parseDouble(), Long.parseLong()
```

■ Use Case: String se number nikalna — bahut common in competitive problems.

17 StringBuilder (MOST IMPORTANT!)

Mutable string class. Use StringBuilder instead of String in loops — much faster. String is immutable so every + creates a new object.

```
StringBuilder sb = new StringBuilder("hello");

sb.append(" world");    // "hello world"

sb.reverse();           // "dlrow olleh"

sb.insert(0, "Hi ");    // insert at index 0

sb.delete(0, 3);        // delete index 0 to 2

sb.charAt(0);           // get char

sb.length();            // get length

sb.toString();          // convert to String
```

■ **Use Case: Loop mein string build karna, reverse karna — String se 100x fast. Always prefer in problems!**

18 Character class methods

Utility methods to check/convert individual char values.

```
char c = 'A';

Character.isLetter(c);      // true

Character.isDigit(c);       // false

Character.isAlphabetic(c);  // true

Character.isWhitespace(c);  // false

Character.toLowerCase(c);   // 'a'

Character.toUpperCase('a'); // 'A'
```

■ **Use Case: Char validate karna — letter hai ya digit — input checking, parsing.**

19 Reverse a String

Java mein Python jaisa [::-1] nahi hota. StringBuilder.reverse() ya loop use karo.

```
// Method 1: StringBuilder (Best)

String rev = new StringBuilder("hello").reverse().toString();

// "olleh"


// Method 2: loop

String s = "hello"; String r = "";
```

```
for(int i = s.length()-1; i >= 0; i--)

    r += s.charAt(i);
```

■ Use Case: Palindrome check — most asked interview question! Always use StringBuilder.reverse().

20 Anagram Check using Arrays.sort()

Sort both strings and compare — simplest anagram solution.

```
import java.util.Arrays;


String s1 = "listen", s2 = "silent";

char[] a = s1.toCharArray();

char[] b = s2.toCharArray();

Arrays.sort(a);

Arrays.sort(b);

boolean isAnagram = Arrays.equals(a, b); // true
```

■ Use Case: Anagram check — most asked! toCharArray() + Arrays.sort() + Arrays.equals().

■ Quick Reference Cheatsheet

Kaam kya karna hai	Java Function	Returns
Size nikalna	`length()`	int
Character access	`charAt(index)`	char
Substring nikalna	`substring(s, e)`	String
Search karna	`indexOf() / contains()`	int / boolean
Compare karna	`equals() / equalsIgnoreCase()`	boolean
Case change	`toUpperCase() / toLowerCase()`	String
Spaces hatana	`trim() / strip()`	String
Replace karna	`replace() / replaceAll()`	String
Todna	`split(delimiter)`	String[]
Jodna	`String.join(delim, ...)`	String
Reverse karna	`StringBuilder.reverse()`	StringBuilder
Char array banana	`toCharArray()`	char[]
Sort karna	`Arrays.sort(charArr)`	void
int to String	`String.valueOf(n)`	String
String to int	`Integer.parseInt(s)`	int
Char validate	`Character.isLetter/isDigit`	boolean
Fast concatenation	`StringBuilder.append()`	StringBuilder

■ Most Asked Interview Problems

Problem	Functions Used	LeetCode #
Palindrome Check	`charAt() / StringBuilder.reverse()`	**#125**
Anagram Check	`toCharArray() + Arrays.sort()`	**#242**
Reverse Words in String	`split() + StringBuilder + join()`	**#151**
Count Vowels	`charAt() + loop + contains()`	**Custom**
First Non-Repeating Char	`indexOf() + lastIndexOf()`	**#387**
String to Integer (atoi)	`charAt() + Character.isDigit()`	**#8**
Remove Duplicates	`indexOf() + StringBuilder`	**Custom**

Caesar Cipher	`charAt() + (char)(ord+shift)`	**Custom**
Valid Parentheses	`charAt() + Stack`	**#20**
Longest Substring	`indexOf() + substring()`	**#3**

■ Pro Tips for Interviews

■ **Tip 1:** Palindrome — pehle lowercase karo, spaces hatao, phir reverse se compare karo.
■ **Tip 2:** Anagram — length same nahi hai toh seedha false return karo, phir sort karo.
■ **Tip 3:** Loop mein string build karna ho toh **StringBuilder** use karo — String nahi!
■ **Tip 4:** Character frequency ke liye int[26] array use karo — HashMap se fast hai.
■ **Tip 5:** ord() jaisa kaam Java mein: **(int)'a'** = 97, **(char)(97)** = 'a'

Made with ❤■ for Java Placement Prep • Practice on LeetCode daily • Consistency = Success ■